

EMCOMM FIELD NODE

A provisioner for offline emergency communications workstations — design, method and limits

Summary

An emergency communications node is a laptop that must keep working after the network it was built on has gone away. Building one by hand is a day of work, undocumented, and different every time — and the differences only surface in the field, where they cost the most.

This project builds that node from one script, in one pass, and then checks what actually landed on disk. It provisions positioning and time, HF and VHF digital modes, offline mapping, an offline reference library, packet and mesh tooling, and software-defined radio, onto a clean Linux Mint install.

What distinguishes it is not the software list. It is the discipline around two questions that field software usually answers badly: **does a check prove what it appears to prove**, and **what is this allowed to touch**. Both are treated as design constraints rather than documentation problems, and both are described here with the failures that produced them.

The problem

A field node is used when infrastructure is unavailable. That inverts several assumptions a normal install rests on:

- **No package server.** Anything not present at deployment time is not available. Offline maps, reference material and codeplugs have to be staged while the network still exists.
- **No time server.** NTP pools are unreachable. A clock that drifts out of a digital mode's timed transmit window makes a station deaf and inaudible with no error message.
- **No second attempt.** An operator discovering a misconfiguration in the field has no way to fix it and often no way to diagnose it.

Manual builds fail these quietly. Two nodes built by the same person a week apart differ, and nobody knows which differences matter until one of them does.

Architecture

A single Python script drives everything. It presents five screens — options, administrator access, run, verification, summary — plus an operating-area screen that appears after the options only when the map step is selected. It carries fifteen independently selectable steps:

Step	What it covers
Callsign / node ID	identity, hostname, passwordless sudo for the RF daemons
System packages	the base stack and Wine initialisation
Slim appliance build	removes desktop extras a field node does not need
QLog + ion2G	station logging and the HF ALE data bridge
Offline map tiles	raster pyramid for the declared operating area
Offline knowledgebase	Kiwix ZIM archives
Reference library	PDF manuals and a loopback-only document server
App profiles	application configuration and the ALE channel plan
Dock-trigger autostart	udev rule and autostart chain for the reference dock — unproven
GPS time source	gpsd and chrony, bound to a declared receiver
Direwolf	AX.25 / APRS software TNC
Meshtastic CLI	LoRa mesh tooling
dump1090	ADS-B aircraft tracking
SatDump	weather satellite imagery
Desktop shortcuts	launchers for what was installed

Steps are independent and not dependency-checked, deliberately. A single step can be re-run in isolation, which is what makes the script usable for development and repair rather than only for first builds.

Administrator access is collected on its own screen **after** the steps are chosen, and is not presented as an option to tick. Provisioning writes to `/etc`, installs packages, enables systemd units and adds a udev rule. It is a requirement of the run, and pretending otherwise would be a lie told by the user interface.

Method

Presence is not capability

The recurring defect class in this project is a check that passes for work that did not happen. It has appeared in enough distinct forms to be treated as the default failure rather than an occasional one:

- A staged configuration file that copied cleanly but substituted nothing — present, correct-looking, and inert.
- A satellite tracking file that existed and contained zero entries, while the row asserting it checked only that the path existed.
- Four green rows attesting that dock automation files were written, on a machine that has never seen a dock.
- A map source registered under a key the installed application does not read, listed in its interface and never drawn.

The response is a rule rather than a fix: **a verification row reads content, not paths**. An observed full run ended with seventy such rows. They confirm that placeholders were substituted, that a declared device path resolves, that a configuration names a card this machine has, that a tile exists beneath the saved map view. Where a row cannot check the thing that matters, it says so in its own text instead of passing quietly.

No row that can only ever warn

A check that can never pass teaches people to skim past warnings, which defeats the screen it lives on. "Never observed keying a radio" is therefore a caveat attached to the row that describes the radio binding, not a row of its own.

Declared, never probed

The provisioner does not search for hardware. An operator names the GPS receiver and the radio interface in configuration files; the provisioner substitutes those values and checks the paths exist.

This is a refusal to be clever, and the reason is specific. **Opening a serial port asserts its control lines**. A radio interface of the type this project targets keys its transmitter from one of those lines. A provisioner that swept `/dev/ttyUSB*` looking for hardware could put a station on the air without anyone asking it to — into an antenna that may not be connected, on a frequency nobody checked, under a callsign that may be a placeholder.

The same rule then pays for itself a second way. A GPS receiver and a radio interface both enumerate as `ttyUSB*`, and which one receives `ttyUSB0` depends on the order they were plugged in. Declared bindings use `/dev/serial/by-id/` paths, which are keyed on vendor, product and serial, and survive that.

Both bindings are excluded from version control, because a `by-id` path carries a device serial number. Samples ship; the filled-in files do not.

Check the version in the field, not the tip

Two separate faults in this project came from reading a dependency's current source while the platform ships an older release.

The offline mapping application is the example. Its tile-source syntax gained brace-style template parameters in one version and the platform ships a version before it, so the sources written were requesting a file whose name was the literal template. Separately, that application renamed its map-list configuration group and moved activation into it — and on the shipped version, every activation key written was ignored, leaving maps listed in the interface and none drawn. Both produced an identical blank screen, which is why the first three explanations were wrong.

The rule now stated in the project's own checklist: **confirm behaviour against the version that ships on the target platform**, not against the project's current source tree.

Data handling

No operator or machine data is committed. Shipped configuration uses substitution tokens filled in at provisioning time. The rule is enforced by verification rows that fail when a token survives, and by a check that a grid square field is empty rather than carrying one captured from whichever machine a profile came from.

This is a rule with history. Real data — a callsign, a grid square, audio device strings, absolute home paths — was once captured into this repository from a working node and required a full history rewrite to remove. The constraints above exist because that happened, not in anticipation of it.

Device bindings that carry serial numbers are excluded from version control by pattern, and the exclusion is asserted by test rather than assumed.

What is proven, and what is not

A project of this kind is only as trustworthy as its account of its own limits. The distinction maintained throughout is between what has been **observed working on hardware** and what has been **written and checked but never exercised**.

Observed on hardware: provisioning completing and reporting honestly; ADS-B tracking live aircraft and releasing its receiver cleanly; weather satellite software building from source with its receiver visible in the recorder and a full tracking set loaded; a kernel driver blacklist surviving a device replug and a reboot; a GPS receiver reaching stratum one within about a minute of a cold boot with no network, twice; offline maps drawing from local tiles with the network pulled.

That last one is worth stating precisely, because it is the kind of claim this section exists to discipline. When it was first recorded, the map sources were listed and the operator selected the layer by hand. The activation keys being written were ones the mapping application's shipped version does not read — it renamed that configuration group in a later release — so nothing was activated, and a listed source is indistinguishable from an active one at a glance. The layer now draws unselected, and that has been observed on a node. The original record was true about the tiles and wrong about the activation, and only a direct before-and-after on one machine separated the two.

Not observed, and stated as such:

- **Dock-triggered autostart has never fired**, on any hardware. The step installs four artifacts and verification confirms all four exist. Whether a dock insertion launches anything is unknown, and no dock is available to the project to test against. It ships selectable and documented as in development, because a convenience feature failing to start is not a node that cannot communicate.
- **No radio has been keyed**. The audio and PTT bindings are declared, substituted and path-checked. Nothing in this project has put a signal on the air.
- **Verification checks the filesystem, not radio frequency**. No automated check covers anything requiring a transmitter or an antenna.

Where a claim is inherited rather than measured — a figure from a datasheet, a hardware identifier from a walkthrough rather than a dump of the real device — it is labelled as such in the source that uses it.

Limits

- **One reference platform**. The stack targets Linux Mint on a specific rugged laptop. Other hardware is untested rather than unsupported.
- **One test bed**. Findings come from a small number of machines operated by one person. Combinations that only appear at scale have not been seen.
- **Steps are not dependency-checked**. Selecting a later step without an earlier one may produce a node that verifies cleanly and does not work. This is a deliberate trade for the ability to re-run a single step.
- **Offline data is operator-supplied**. Map tiles, reference archives and application profiles are fetched or staged by the operator. The repository ships the mechanism, not the content.

Licensing

Released under the **GNU General Public License v3.0 or later**. The licence text ships with every release archive; it is deliberately not excluded from the packaged artifact.

Some capabilities require an amateur radio licence to operate legally on the air. Receiving is unrestricted; transmitting is not, and the software says so at the point where a callsign is configured rather than only in documentation.

Closing

The engineering claim this project makes is narrow and worth stating plainly: a field node should be reproducible, and a tool that builds one should be honest about what it has and has not established.

Most of the design decisions recorded here were bought with a failure. The rules read as fussy in isolation — read content not paths, never probe a serial port, check the shipped version, do not write a row that can only warn — and each of them exists because the alternative already cost a day of testing or shipped something quietly wrong.